

Лабораторная работа № 5. Эмуляция и измерение потерь пакетов в глобальных сетях задержек в глобальных сетях

Абакумова Олеся Максимовна, НФИбд-02-22

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	Запуск лабораторной топологии	7
3.2	Интерактивные эксперименты	10
3.2.1	Добавление потери пакетов на интерфейс, подключённый к эмулируемой глобальной сети	10
3.2.2	Добавление значения корреляции для потери пакетов в эмулируемой глобальной сети	13
3.2.3	Добавление повреждения пакетов в эмулируемой глобальной сети	14
3.2.4	Добавление переупорядочивания пакетов в интерфейс подключения к эмулируемой глобальной сети	15
3.2.5	Добавление дублирования пакетов в интерфейс подключения к эмулируемой глобальной сети	17
3.3	Воспроизведение экспериментов	19
3.3.1	Предварительная подготовка	19
3.3.2	Добавление потери пакетов на интерфейс, подключённый к эмулируемой глобальной сети	20
3.4	Задание для самостоятельной работы	23
4	Выводы	26

Список иллюстраций

3.1	Запуск виртуальной машины	7
3.2	Исправление прав запуска X-соединения	8
3.3	Задание простейшей топологии	8
3.4	Информация на хосте h1	9
3.5	Информация на хосте h2	9
3.6	Проверка подключения между хостами	9
3.7	Добавление 10% потерь пакетов	10
3.8	Проверка соединения	11
3.9	Добавление 10% потерь пакетов на хосте h2	11
3.10	Проверка, что соединение имеет большой процент потерь	12
3.11	Восстановление конфигураций	12
3.12	Проверка, что соединение не имеет явной потери	13
3.13	Добавление коэффициента потери пакетов 50%	13
3.14	Проверка, что имеются потери пакетов	14
3.15	Восстановление для узла h1 конфигурации по умолчанию и проверка	14
3.16	Добавление повреждения пакетов 0,01%	15
3.17	Проверка конфигурации с помощью iPerf3	15
3.18	Восстановление конфигурации h1	15
3.19	Добавление правила	16
3.20	Проверка после добавления правила	17
3.21	Задание правила с дублированием	18
3.22	Проверка на наличие дубликатов	19
3.23	Создание каталога	19
3.24	Создание simple-drop	20
3.25	Создание lab_netem_ii.py	20
3.26	Коррекция скрипта	21
3.27	Создание Makefile	21
3.28	Запуск скрипта	22
3.29	Коррекция скрипта для ping.dat	22
3.30	Наличие ping.dat	23
3.31	Содержимое ping.dat после повторного запуска скрипта	23
3.32	Скрипт для самостоятельного задания	24
3.33	Запуск обновленного скрипта	24
3.34	Пример одного из файлов воспроизводимого эксперимента	25

Список таблиц

1 Цель работы

Основной целью работы является получение навыков проведения интерактивных экспериментов в среде Mininet по исследованию параметров сети, связанных с потерей, дублированием, изменением порядка и повреждением пакетов при передаче данных. Эти параметры влияют на производительность протоколов и сетей

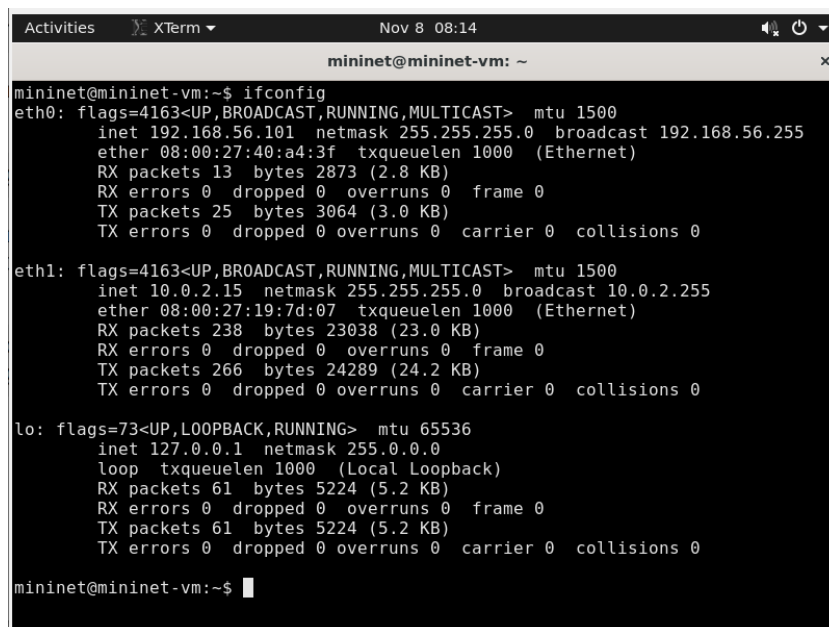
2 Задание

1. Задайте простейшую топологию, состоящую из двух хостов и коммутатора с назначенной по умолчанию mininet сетью 10.0.0.0/8.
2. Проведите интерактивные эксперименты по исследованию параметров сети, связанных с потерей, дублированием, изменением порядка и повреждением пакетов при передаче данных.
3. Реализуйте воспроизводимый эксперимент по добавлению правила отбрасывания пакетов в эмулируемой глобальной сети. На экран выведите сводную информацию о потерянных пакетах.
4. Самостоятельно реализуйте воспроизводимые эксперименты по исследованию параметров сети, связанных с потерей, изменением порядка и повреждением пакетов при передаче данных. На экран выведите сводную информацию о потерянных пакетах.

3 Выполнение лабораторной работы

3.1 Запуск лабораторной топологии

Запускаем виртуальную машину. Здесь ждет легкая неожиданность, но это не помешало выполнить лабораторную работу (рис. 3.1):



```
mininet@mininet-vm:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.101 netmask 255.255.255.0 broadcast 192.168.56.255
    ether 08:00:27:40:a4:3f txqueuelen 1000 (Ethernet)
    RX packets 13 bytes 2873 (2.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 25 bytes 3064 (3.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    ether 08:00:27:19:7d:07 txqueuelen 1000 (Ethernet)
    RX packets 238 bytes 23038 (23.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 266 bytes 24289 (24.2 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 61 bytes 5224 (5.2 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 61 bytes 5224 (5.2 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

mininet@mininet-vm:~$
```

Рис. 3.1: Запуск виртуальной машины

В виртуальной машине mininet при необходимости исправим права запуска X-соединения. Скопируем значение куки (MIT magic cookie) своего пользователя mininet в файл для пользователя root(рис. 3.2):

```

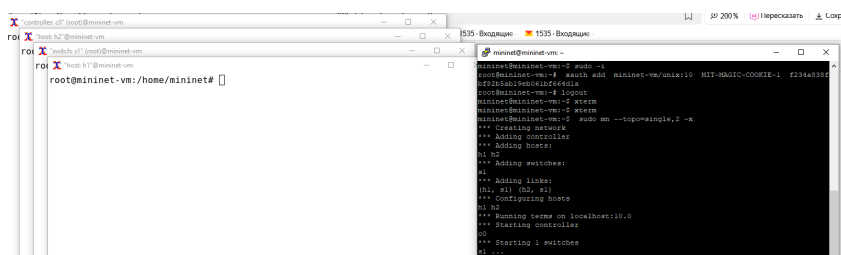
mininet@mininet-vm:~$ xauth list $DISPLAY
mininet-vm/unix:10 MIT-MAGIC-COOKIE-1 f234a838fbf82b5ab19eb061bf664d1a
mininet@mininet-vm:~$ sudo -i
root@mininet-vm:~# xauth add mininet-vm/unix:10 MIT-MAGIC-COOKIE-1 f234a838fbf82b5ab19eb061bf664d1a
root@mininet-vm:~# logout
mininet@mininet-vm:~$ xterm

```

Рис. 3.2: Исправление прав запуска X-соединения

После выполнения этих действий графические приложения должны запускаться под пользователем mininet.

Зададим простейшую топологию, состоящую из двух хостов и коммутатора с назначенной по умолчанию mininet сетью 10.0.0.0/8 (рис. 3.3):



```

mininet@mininet-vm:~$ xterm
mininet@mininet-vm:~$ mininet
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1), (s1, h2)
*** Configuring hosts
h1 h2
*** Running tests on localhost:10.0
*** Starting controller
c0
*** Starting 1 switches
s1...

```

Рис. 3.3: Задание простейшей топологии

После введения этой команды запустятся терминалы двух хостов, коммутатора и контроллера. Терминалы коммутатора и контроллера можно закрыть.

На хостах h1 и h2 введем команду `ifconfig`, чтобы отобразить информацию, относящуюся к их сетевым интерфейсам и назначенным им IP-адресам. В дальнейшем при работе с NETEM и командой `tc` будут использоваться интерфейсы `h1-eth0` и `h2-eth0` (рис. 3.4):


```

root@mininet-virtual-machine:~# ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.1 netmask 255.0.0.0 broadcast 10.255.255.255
    ether 6a:1d:55:e2:06:54 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1192 bytes 285088 (285.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1192 bytes 285088 (285.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-virtual-machine:~#

```

Рис. 3.4: Информация на хосте h1

```

root@mininet-virtual-machine:~# ifconfig
h2-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.2 netmask 255.0.0.0 broadcast 10.255.255.255
    ether fe:66:2a:79:b1:1e txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

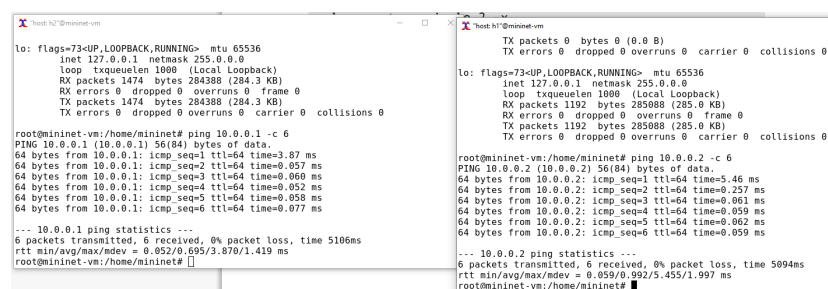
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1474 bytes 284388 (284.3 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1474 bytes 284388 (284.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-virtual-machine:~#

```

Рис. 3.5: Информация на хосте h2

Проверим подключение между хостами h1 и h2 с помощью команды ping с параметром -c 6 (рис. 3.6):



```

root@mininet-virtual-machine:~# ifconfig
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1474 bytes 284388 (284.3 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1474 bytes 284388 (284.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-virtual-machine:~# ping 10.0.0.1 -c 6
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=3.87 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.057 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.060 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.052 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.059 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.077 ms
--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5106ms
rtt min/avg/max/mdev = 0.052/0.095/3.870/1.419 ms
root@mininet-virtual-machine:~#

root@mininet-virtual-machine:~# ifconfig
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1192 bytes 285088 (285.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1192 bytes 285088 (285.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-virtual-machine:~# ping 10.0.0.2 -c 6
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=5.46 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.257 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.061 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.059 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.059 ms
--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5094ms
rtt min/avg/max/mdev = 0.059/0.092/5.455/1.997 ms
root@mininet-virtual-machine:~#

```

Рис. 3.6: Проверка подключения между хостами

Минимальное RTT: 0.052 мс, среднее RTT: 0.095 мс, максимальное RTT: 3.870 мс, стандартное отклонение RTT: 1.419 мс для 10.0.0.1. Для 10.0.0.2: минималь-

ное RTT: 0.059 мс, среднее RTT: 0.992 мс, максимальное RTT: 5.435 мс, стандартное отклонение RTT: 1.997 мс. Потерь данных в обоих случаях нет (0% потерь пакетов).

3.2 Интерактивные эксперименты

3.2.1 Добавление потери пакетов на интерфейс, подключённый к эмулируемой глобальной сети

Пакеты могут быть потеряны в процессе передачи из-за таких факторов, как битовые ошибки и перегрузка сети. Скорость потери данных часто измеряется как процентная доля потерянных пакетов по отношению к количеству отправленных пакетов. На хосте h1 добавьте 10% потерь пакетов к интерфейсу h1-eth0 (рис. 3.7):

```
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem loss 10%
root@mininet-vm:/home/mininet# █
```

Рис. 3.7: Добавление 10% потерь пакетов

Здесь:

- `sudo`: выполнить команду с более высокими привилегиями;
- `tc`: вызвать управление трафиком Linux;
- `qdisc`: изменить дисциплину очередей сетевого планировщика;
- `add`: создать новое правило;
- `dev h1-eth0`: указать интерфейс, на котором будет применяться правило;
- `netem`: использовать эмулятор сети;
- `loss 10%`: 10% потерь пакетов.

Проверим, что на соединении от хоста h1 к хосту h2 имеются потери пакетов, используя команду `ping` с параметром `-c 100` с хоста h1. Параметр `-c` указывает общее количество пакетов для отправки. Обратим внимание на значения `icmp_seq`. Некоторые номера последовательности отсутствуют из-за потери пакетов. В сводном отчёте `ping` сообщает о проценте потерянных пакетов после завершения передачи (рис. 3.8):

```
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 100
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=3.46 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.531 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.160 ms
█
```

Рис. 3.8: Проверка соединения

Для эмуляции глобальной сети с потерей пакетов в обоих направлениях необходимо к соответствующему интерфейсу на хосте h2 также добавить 10% потерь пакетов (рис. 3.9):

```
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h2-eth0 root netem loss 10
1%
root@mininet-vm:/home/mininet# █
```

Рис. 3.9: Добавление 10% потерь пакетов на хосте h2

Проверим, что соединение между хостом h1 и хостом h2 имеет больший процент потерянных данных (10% от хоста h1 к хосту h2 и 10% от хоста h2 к хосту h1), повторив команду `ping` с параметром `-c 100` на терминале хоста h1 (рис. 3.10):

```

X "host: h1" @mininet-vm
64 bytes from 10.0.0.2: icmp_seq=75 ttl=64 time=0.060 ms
64 bytes from 10.0.0.2: icmp_seq=77 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=78 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=79 ttl=64 time=0.094 ms
64 bytes from 10.0.0.2: icmp_seq=81 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=82 ttl=64 time=0.051 ms
64 bytes from 10.0.0.2: icmp_seq=83 ttl=64 time=0.059 ms
64 bytes from 10.0.0.2: icmp_seq=84 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=85 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=87 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=88 ttl=64 time=0.064 ms
64 bytes from 10.0.0.2: icmp_seq=89 ttl=64 time=0.066 ms
64 bytes from 10.0.0.2: icmp_seq=91 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=94 ttl=64 time=0.065 ms
64 bytes from 10.0.0.2: icmp_seq=95 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=96 ttl=64 time=0.064 ms
64 bytes from 10.0.0.2: icmp_seq=97 ttl=64 time=0.076 ms
64 bytes from 10.0.0.2: icmp_seq=98 ttl=64 time=0.080 ms
64 bytes from 10.0.0.2: icmp_seq=99 ttl=64 time=0.081 ms

--- 10.0.0.2 ping statistics ---
100 packets transmitted, 78 received, 22% packet loss, time 101359ms
rtt min/avg/max/mdev = 0.051/0.084/0.885/0.109 ms
root@mininet-vm:/home/mininet#

```

Рис. 3.10: Проверка, что соединение имеет большой процент потерь

22% потерь пакетов

Восстановим конфигурацию по умолчанию, удалив все правила, применённые к сетевому планировщику соответствующего интерфейса. Для отправителя h1 (рис. 3.11):

```

отправителю X "host: h1" @mininet-vm
64 bytes from 10.0.0.2: icmp_seq=77 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=78 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=79 ttl=64 time=0.094 ms
X "host: h2" @mininet-vm
64 bytes from 10.0.0.2: icmp_seq=81 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=82 ttl=64 time=0.051 ms
64 bytes from 10.0.0.2: icmp_seq=83 ttl=64 time=0.059 ms
64 bytes from 10.0.0.2: icmp_seq=84 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=85 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=87 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=88 ttl=64 time=0.064 ms
64 bytes from 10.0.0.2: icmp_seq=89 ttl=64 time=0.066 ms
64 bytes from 10.0.0.2: icmp_seq=91 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=94 ttl=64 time=0.065 ms
64 bytes from 10.0.0.2: icmp_seq=95 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=96 ttl=64 time=0.064 ms
64 bytes from 10.0.0.2: icmp_seq=97 ttl=64 time=0.076 ms
64 bytes from 10.0.0.2: icmp_seq=98 ttl=64 time=0.080 ms
64 bytes from 10.0.0.2: icmp_seq=99 ttl=64 time=0.081 ms

--- 10.0.0.2 ping statistics ---
100 packets transmitted, 78 received, 22% packet loss, time 101359ms
rtt min/avg/max/mdev = 0.051/0.084/0.885/0.109 ms
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet#
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h2-eth0 root netem
root@mininet-vm:/home/mininet#

```

Рис. 3.11: Восстановление конфигураций

Убедимся, что соединение от хоста h1 к хосту h2 не имеет явной потери (рис. 3.12):

```
host: h1@mininet-vm
64 bytes from 10.0.0.2: icmp_seq=83 ttl=64 time=0.064 ms
64 bytes from 10.0.0.2: icmp_seq=84 ttl=64 time=0.073 ms
64 bytes from 10.0.0.2: icmp_seq=85 ttl=64 time=0.094 ms
64 bytes from 10.0.0.2: icmp_seq=86 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=87 ttl=64 time=0.058 ms
64 bytes from 10.0.0.2: icmp_seq=88 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=89 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=90 ttl=64 time=0.058 ms
64 bytes from 10.0.0.2: icmp_seq=91 ttl=64 time=0.061 ms
64 bytes from 10.0.0.2: icmp_seq=92 ttl=64 time=0.060 ms
64 bytes from 10.0.0.2: icmp_seq=93 ttl=64 time=0.067 ms
64 bytes from 10.0.0.2: icmp_seq=94 ttl=64 time=0.060 ms
64 bytes from 10.0.0.2: icmp_seq=95 ttl=64 time=0.047 ms
64 bytes from 10.0.0.2: icmp_seq=96 ttl=64 time=0.065 ms
64 bytes from 10.0.0.2: icmp_seq=97 ttl=64 time=0.060 ms
64 bytes from 10.0.0.2: icmp_seq=98 ttl=64 time=0.086 ms
64 bytes from 10.0.0.2: icmp_seq=99 ttl=64 time=0.068 ms
64 bytes from 10.0.0.2: icmp_seq=100 ttl=64 time=0.065 ms

--- 10.0.0.2 ping statistics ---
100 packets transmitted, 100 received, 0% packet loss, time 101340ms
rtt min/avg/max/mdev = 0.042/0.102/3.195/0.317 ms
root@mininet-vm:/home/mininet#
root@mininet-vm:/home/mininet#
```

Рис. 3.12: Проверка, что соединение не имеет явной потери

3.2.2 Добавление значения корреляции для потери пакетов в эмулируемой глобальной сети

Добавим на интерфейсе узла h1 коэффициент потери пакетов 50% (такой высокий уровень потери пакетов маловероятен), и каждая последующая вероятность зависит на 50% от последней (рис. 3.13):

```
root@mininet-vm:/home/mininet#
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem loss 50
% 50%
root@mininet-vm:/home/mininet#
```

Рис. 3.13: Добавление коэффициента потери пакетов 50%

Проверим, что на соединении от хоста h1 к хосту h2 имеются потери пакетов, используя команду ping с параметром -c 50 с хоста h1 (рис. 3.14):

```
host: h1" @mininet-vm
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.581 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.731 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.081 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.060 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.065 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.070 ms
64 bytes from 10.0.0.2: icmp_seq=25 ttl=64 time=0.263 ms
64 bytes from 10.0.0.2: icmp_seq=26 ttl=64 time=0.068 ms
64 bytes from 10.0.0.2: icmp_seq=27 ttl=64 time=0.061 ms
64 bytes from 10.0.0.2: icmp_seq=28 ttl=64 time=0.069 ms
64 bytes from 10.0.0.2: icmp_seq=30 ttl=64 time=0.060 ms
64 bytes from 10.0.0.2: icmp_seq=34 ttl=64 time=0.059 ms
64 bytes from 10.0.0.2: icmp_seq=36 ttl=64 time=0.066 ms
64 bytes from 10.0.0.2: icmp_seq=41 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=42 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=43 ttl=64 time=0.079 ms
64 bytes from 10.0.0.2: icmp_seq=44 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=49 ttl=64 time=0.067 ms

--- 10.0.0.2 ping statistics ---
50 packets transmitted, 20 received, 60% packet loss, time 50139ms
rtt min/avg/max/mdev = 0.059/0.267/2.714/0.589 ms
root@mininet-vm:/home/mininet#
```

Рис. 3.14: Проверка, что имеются потери пакетов

60% потеря пакетов

Восстановим для узла h1 конфигурацию по умолчанию, удалив все правила, применённые к сетевому планировщику соответствующего интерфейса (рис. 3.15):

```
rtt min/avg/max/mdev = 0.059/0.267/2.714/0.589 ms
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 10
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=2.86 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.515 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.189 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.061 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.073 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.057 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.061 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.059 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.057 ms

--- 10.0.0.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9189ms
rtt min/avg/max/mdev = 0.057/0.399/2.860/0.831 ms
root@mininet-vm:/home/mininet#
```

Рис. 3.15: Восстановление для узла h1 конфигурации по умолчанию и проверка

3.2.3 Добавление повреждения пакетов в эмулируемой глобальной сети

Добавим на интерфейсе узла h1 0,01% повреждения пакетов (рис. 3.16):

```
root@mininet-virtual-machine:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem corrupt
0.01%
root@mininet-virtual-machine:/home/mininet#
```

Рис. 3.16: Добавление повреждения пакетов 0,01%

Проверим конфигурацию с помощью инструмента iPerf3 для проверки повторных передач(рис. 3.17):

```

root@mininet-virtual-machine:/home/mininet# iPerf3 -c 10.0.0.2
Connecting to host 10.0.0.2, port 5201
[ 7] local 10.0.0.1 port 40112 connected to 10.0.0.2 port 5201
[ ID] Interval      Transfer     Bitrate      Retr  Cwnd
[ 7] 0.00-1.00 sec  923 MBytes  7.75 Gbits/sec  0    8.10 MBytes
[ 7] 1.00-2.00 sec  1.11 GBytes  9.57 Gbits/sec  3    2.83 MBytes
[ 7] 2.00-3.00 sec  1.45 GBytes  12.4 Gbits/sec  2    1.73 MBytes
[ 7] 3.00-4.00 sec  1.61 GBytes  13.9 Gbits/sec  1    1.33 MBytes
[ 7] 4.00-5.00 sec  1.57 GBytes  13.5 Gbits/sec  2    765 KBytes
[ 7] 5.00-6.00 sec  1.60 GBytes  13.7 Gbits/sec  3    615 KBytes
[ 7] 6.00-7.00 sec  1.54 GBytes  13.3 Gbits/sec  0    1.70 MBytes
[ 7] 7.00-8.00 sec  1.61 GBytes  13.8 Gbits/sec  4    571 KBytes
[ 7] 8.00-9.00 sec  1.47 GBytes  12.6 Gbits/sec  2    724 KBytes
[ 7] 9.00-10.00 sec 1.59 GBytes  13.7 Gbits/sec  2    1.58 MBytes
[ ID] Interval      Transfer     Bitrate      Retr
[ 7] 0.00-10.01 sec 14.4 GBytes  12.4 Gbits/sec  19
iperf Done.
root@mininet-virtual-machine:/home/mininet#

root@mininet-virtual-machine:/home/mininet# iPerf3 -s
warning: this system does not seem to support IPv6 - trying IPv4
Server listening on 5201
Accepted connection from 10.0.0.1, port 40110
[ ID] Interval      Transfer     Bitrate
[ 7] 0.00-1.00 sec  911 MBytes  7.64 Gbits/sec
[ 7] 1.00-2.00 sec  1.11 GBytes  9.57 Gbits/sec
[ 7] 2.00-3.00 sec  1.43 GBytes  12.3 Gbits/sec
[ 7] 3.00-4.00 sec  1.60 GBytes  13.7 Gbits/sec
[ 7] 4.00-5.00 sec  1.57 GBytes  13.5 Gbits/sec
[ 7] 5.00-6.00 sec  1.59 GBytes  13.7 Gbits/sec
[ 7] 6.00-7.00 sec  1.54 GBytes  13.2 Gbits/sec
[ 7] 7.00-8.00 sec  1.61 GBytes  13.8 Gbits/sec
[ 7] 8.00-9.00 sec  1.48 GBytes  12.7 Gbits/sec
[ 7] 9.00-10.00 sec 1.59 GBytes  13.6 Gbits/sec
[ 7] 10.00-10.01 sec 9.75 MBytes  6.37 Gbits/sec
[ ID] Interval      Transfer     Bitrate
[ 7] 0.00-10.01 sec 14.4 GBytes  12.4 Gbits/sec
Server listening on 5201

```

Рис. 3.17: Проверка конфигурации с помощью iPerf3

Восстановим конфигурацию на h1(рис. 3.18):

```
iperf Done.
root@mininet-virtual-machine:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-virtual-machine:/home/mininet#
```

Рис. 3.18: Восстановление конфигурации h1

3.2.4 Добавление переупорядочивания пакетов в интерфейс подключения к эмулируемой глобальной сети

Добавим на интерфейсе узла h1 следующее правило (рис. 3.19):

```
host: h1" @mininet-vm
RX packets 970  bytes 241612 (241.6 KB)
RX errors 0  dropped 0  overruns 0  frame 0
TX packets 970  bytes 241612 (241.6 KB)
TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0

root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 10
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=2.97 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.368 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.058 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.064 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.060 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.073 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.131 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.060 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.067 ms

--- 10.0.0.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9191ms
rtt min/avg/max/mdev = 0.058/0.391/2.967/0.863 ms
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 1
0ms reorder 25% 50%
root@mininet-vm:/home/mininet#
```

Рис. 3.19: Добавление правила

Здесь 25% пакетов (со значением корреляции 50%) будут отправлены немедленно, а остальные 75% будут задержаны на 10 мс.

Проверим, что на соединении от хоста h1 к хосту h2 имеются потери пакетов, используя команду ping с параметром -c 20 с хоста h1. Убедимся, что часть пакетов не будут иметь задержки (один из четырех, или 25%), а последующие несколько пакетов будут иметь задержку около 10 миллисекунд (три из четырех, или 75%). При необходимости повторим тест (рис. 3.20):


```
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 20
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=10.6 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=10.6 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=10.3 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=10.6 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=10.6 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=10.3 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=10.4 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=10.3 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=10.3 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=10.3 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=10.2 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=0.061 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=10.4 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=10.7 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19088ms
rtt min/avg/max/mdev = 0.061/9.462/10.721/3.138 ms
root@mininet-vm:/home/mininet#
```

Рис. 3.20: Проверка после добавления правила

0% потерь пакетов

Восстановим конфигурацию.

3.2.5 Добавление дублирования пакетов в интерфейс подключения к эмулируемой глобальной сети

Для интерфейса узла h1 зададим правило с дублированием 50% пакетов (т.е. 50% пакетов должны быть получены дважды) (рис. 3.21):

```
host: h1" @mininet-vm
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=10.4 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=10.7 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19088ms
rtt min/avg/max/mdev = 0.061/9.462/10.721/3.138 ms
root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 20
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=10.3 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=10.2 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=10.6 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=10.3 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=10.8 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=10.8 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.073 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=10.5 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=11.2 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=10.6 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=10.7 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=10.9 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19057ms
rtt min/avg/max/mdev = 0.073/10.128/11.161/2.316 ms
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem duplica
te 50%
root@mininet-vm:/home/mininet#
```

Рис. 3.21: Задание правила с дублированием

Проверим, что на соединении от хоста h1 к хосту h2 имеются дублированные пакеты, используя команду `ping` с параметром `-c 20` с хоста h1. Дубликаты пакетов помечаются как DUP!. Измеренная скорость дублирования пакетов будет приближаться к настроенной скорости по мере выполнения большего количества попыток (рис. 3.22):

```

root@mininet-vm:/home/mininet# ping 10.0.0.2 -c 20
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=3.37 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.492 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.226 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=1.29 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.062 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.063 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.061 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.069 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.069 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.079 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.067 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.067 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.058 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=0.099 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=0.099 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=0.066 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=0.067 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=0.068 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=0.101 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=0.102 ms (DUP!)
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=0.074 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, +7 duplicates, 0% packet loss, time 19383ms
rtt min/avg/max/mdev = 0.058/0.260/3.373/0.655 ms
root@mininet-vm:/home/mininet#

```

Рис. 3.22: Проверка на наличие дубликатов

Восстановим конфигурацию.

3.3 Воспроизведение экспериментов

3.3.1 Предварительная подготовка

Для каждого воспроизводимого эксперимента exрname создадим свой каталог, в котором будут размещаться файлы эксперимента (рис. 3.23):

```

mininet@mininet-vm:~$ mkdir -p ~/work/lab_netem_ii/exрname
mininet@mininet-vm:~$

```

Рис. 3.23: Создание каталога

Здесь exрname может принимать значения simple-drop, correlation-drop и т.п.

3.3.2 Добавление потери пакетов на интерфейс, подключённый к эмулируемой глобальной сети

С помощью API Mininet воспроизведем эксперимент по добавлению потери пакетов для интерфейса хоста, подключающегося к эмулируемой глобальной сети. В виртуальной среде mininet в своём рабочем каталоге с проектами создадим каталог simple-drop и перейдем в него (рис. 3.24):

```
mininet@mininet-vm:~$ mkdir -p ~/work/lab_netem_ii/simple-drop
mininet@mininet-vm:~$ cd ~/work/lab_netem_ii/simple-drop
mininet@mininet-vm:~/work/lab_netem_ii/simple-drop$
```

Рис. 3.24: Создание simple-drop

Создадим скрипт для эксперимента lab_netem_ii.py (рис. 3.25):

```
GNU nano 4.8 lab_netem_ii.py
#!/usr/bin/env python

"""
Simple experiment.
Output: ping.dat
"""

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info
import time

def emptyNet():
    """Create an empty network and add nodes to it."""
    net = Mininet( controller=Controller,
                  waitConnected=True )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )

    info( '*** Adding switch\n' )
    s1 = net.addSwitch( 's1' )

    info( '*** Creating links\n' )
    net.addLink( h1, s1 )
    net.addLink( h2, s1 )

    info( '*** Starting network\n' )
    net.start()

    info( '*** Set delay\n' )
    h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem loss
~ 10%' )
    h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem loss
~ 10%' )

    time.sleep(10) # Wait 10 seconds

    info( '*** Ping\n' )
    h1.cmdPrint( 'ping -c 100', h2.IP(), '| grep "time=" |
~ awk "{print $5, $7}" | sed -e "s/time=//g" -e
~ "s/icmp_seq=//g" > ping.dat' )

    info( '*** Stopping network\n' )
    net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()
```

Рис. 3.25: Создание lab_netem_ii.py

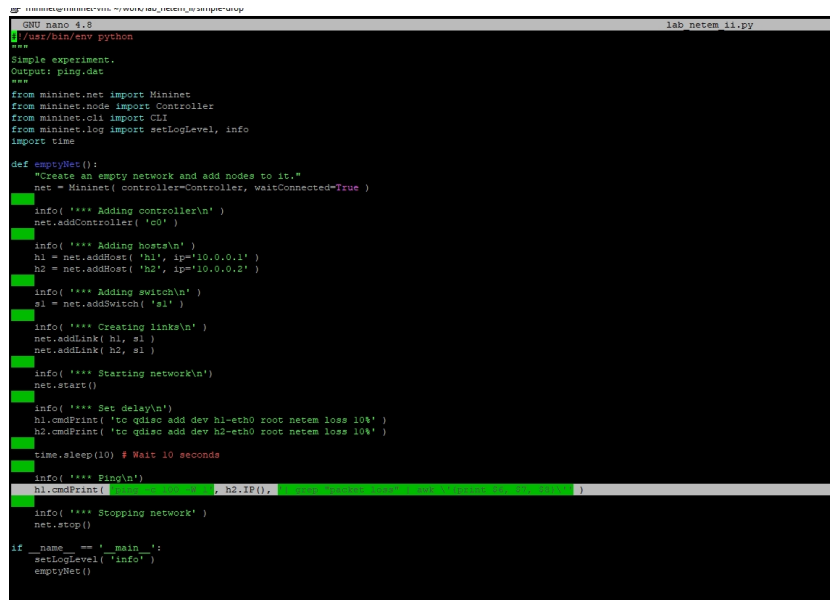
В этих строках задается значение потери пакетов для интерфейса хоста:

```
h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem loss
~ 10%' )
```

```
h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem loss
```

```
~ 10%'
```

Скорректируем скрипт так, чтобы на экран или в отдельный файл выводилась информация о потерях пакетов (рис. 3.26):



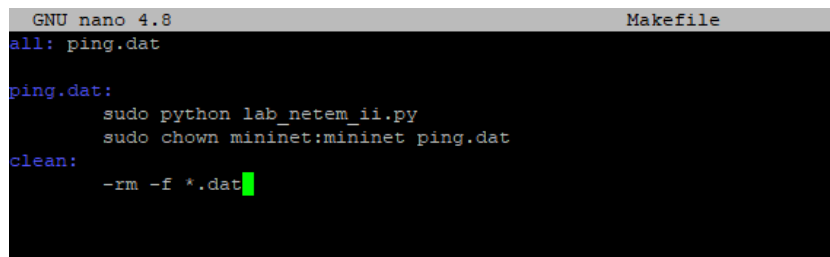
```
GNU nano 4.8 lab_netem.py
#!/usr/bin/env python
'''
Simple experiment.
Output: ping.dat
'''
from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info
import time

def emptyNet():
    """Create an empty network and add nodes to it."""
    net = Mininet( controller=Controller, waitConnected=True )
    info( '*** Adding controller\n' )
    net.addController( 'c0' )
    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )
    info( '*** Adding switches\n' )
    s1 = net.addSwitch( 's1' )
    info( '*** Creating links\n' )
    net.addLink( h1, s1 )
    net.addLink( h2, s1 )
    info( '*** Starting network\n' )
    net.start()
    info( '*** Set delay\n' )
    h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem loss 10%' )
    h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem loss 10%' )
    time.sleep(10) # Wait 10 seconds
    info( '*** Ping\n' )
    h1.cmdPrint( 'ping -c 1 -i 0.1 h2', h2.IP() )
    info( '*** Stopping network\n' )
    net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()
```

Рис. 3.26: Коррекция скрипта

Создадим Makefile для управления процессом проведения эксперимента (рис. 3.27):



```
GNU nano 4.8 Makefile
all: ping.dat

ping.dat:
    sudo python lab_netem.py
    sudo chown mininet:mininet ping.dat

clean:
    -rm -f *.dat
```

Рис. 3.27: Создание Makefile

Запустим скрипт (рис. 3.28):

```

mininet@mininet-vm:~/work/lab_netem_i/simple-drop$ make
sudo python lab_netem_i.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Set delay
*** h1 : ('tc qdisc add dev h1-eth0 root netem loss 10%',)
*** h2 : ('tc qdisc add dev h2-eth0 root netem loss 10%',)
*** Ping
*** h1 : ('ping -c 100 -W 1', '10.0.0.2', '| grep "packet loss" | awk \'{print $6, $7, $8}\'}')
18% packet loss,
*** Stopping network*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done

```

Рис. 3.28: Запуск скрипта

Также помимо вывода например потери пакетов на экран можно сделать вывод в файл. Рассмотрим и такой вариант. Вот как я скорректировала файл (рис. 3.29):

```

@ mininet@mininet-vm:~/work/lab_netem_i/simple-drop
GNU nano 4.8 lab_netem_i.py
#!/usr/bin/env python
***
Simple experiment.
Output: ping.dat
***
from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info
import time

def emptyNet():
    "Create an empty network and add nodes to it."
    net = Mininet(controller=Controller, waitConnected=True)
    info( '*** Adding controller\n' )
    net.addController( 'c0' )
    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )
    info( '*** Adding switch\n' )
    s1 = net.addSwitch( 's1' )
    info( '*** Creating links\n' )
    net.addLink( h1, s1 )
    net.addLink( h2, s1 )
    info( '*** Starting network\n' )
    net.start()
    info( '*** Set delay\n' )
    h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem loss 10%' )
    h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem loss 10%' )
    time.sleep(10) # Wait 10 seconds
    info( '*** Ping\n' )
    h1.cmdPrint( 'ping -c 100 -W 1 10.0.0.2 | grep "packet loss" | awk \'{print $6, $7, $8}\'}' )
    info( '*** Stopping network\n' )
    net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()

```

Рис. 3.29: Коррекция скрипта для ping.dat

Ping.dat (рис. 3.30)теперь есть в каталоге и его можно посмотреть (рис. 3.31):

```
mininet@mininet-vm:~/work/lab_netem_ii/simple-drop$ ls
lab_netem_ii.py  Makefile  ping.dat
```

Рис. 3.30: Наличие ping.dat

```
lab_netem_ii.py Makefile ping.dat
mininet@mininet-vm:~/work/lab_netem_ii/simple-drop$ cat ping.dat
1013 1
1.75 2
0.263 3
0.056 4
0.077 5
0.067 6
0.080 7
0.069 8
0.083 9
0.062 10
0.061 11
0.062 13
0.062 15
0.064 17
0.063 18
0.077 19
0.061 20
0.072 21
0.062 22
0.062 24
0.073 25
0.049 26
0.064 27
```

Рис. 3.31: Содержимое ping.dat после повторного запуска скрипта

3.4 Задание для самостоятельной работы

Самостоятельно реализуйте воспроизводимые эксперименты по исследованию параметров сети, связанных с потерей, изменением порядка и повреждением пакетов при передаче данных (рис. 3.32):

```

GNU nano 4.8                                lab_netem_ii.py
#!/usr/bin/env python

from mininet.net import Mininet
from mininet.node import Controller
from mininet.log import setLogLevel, info
import time
import os

def cleanup():
    os.system('sudo ip link delete h1-eth0 2>/dev/null')
    os.system('sudo ip link delete h2-eth0 2>/dev/null')
    os.system('sudo ip link delete s1-eth1 2>/dev/null')
    os.system('sudo ip link delete s1-eth2 2>/dev/null')
    os.system('sudo ovs-vsctl del-br s1 2>/dev/null')

def emptyNet():
    cleanup()

    net = Mininet(controller=Controller, waitConnected=True)
    net.addController('c0')
    h1 = net.addHost('h1', ip='10.0.0.1')
    h2 = net.addHost('h2', ip='10.0.0.2')
    s1 = net.addSwitch('s1')
    net.addLink(h1, s1)
    net.addLink(h2, s1)
    net.start()

    experiments = [
        ('loss_5', 'loss 5%'),
        ('loss_10', 'loss 10%'),
        ('loss_20', 'loss 20%'),
        ('reorder_25', 'delay 50ms reorder 25%'),
        ('reorder_50', 'delay 100ms reorder 50%'),
        ('corrupt_5', 'corrupt 5%'),
        ('corrupt_10', 'corrupt 10%'),
        ('combined', 'loss 10% delay 100ms corrupt 5%')
    ]

    for exp_name, tc_cmd in experiments:
        h1.cmd('f' to qdisc del dev h1-eth0 root 2>/dev/null')
        h2.cmd('f' to qdisc del dev h2-eth0 root 2>/dev/null')

        h1.cmd('f' to qdisc add dev h1-eth0 root netem (tc_cmd)')
        h2.cmd('f' to qdisc add dev h2-eth0 root netem (tc_cmd)')

        time.sleep(2)

        h1.cmd('f' ping -c 50 -W 1 (h2.IP()) > stats (exp_name).log 2>&1')
        h1.cmd('f' grep "time=" stats (exp_name).log | awk '{print $7, $5}}' | sed -e "s/time=//g" -e "s/icmp_seq//g" > ping_(exp_name).dat')

        result = h1.cmd('f' grep "packet loss" stats (exp_name).log')
        info('f' (exp_name): (%s)')

    net.stop()

if __name__ == '__main__':
    setLogLevel('info')
    emptyNet()

```

Рис. 3.32: Скрипт для самостоятельного задания

Запустим обновленный скрипт (рис. 3.33):

```

mininet@mininet-vm:~/work/lab_netem_ii/simple-drop$ make clean
rm -f *.dat
mininet@mininet-vm:~/work/lab_netem_ii/simple-drop$ make
sudo python lab_netem_ii.py
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
loss 5: 50 packets transmitted, 37 received, 26% packet loss, time 50134ms
loss 10: 50 packets transmitted, 41 received, 18% packet loss, time 50182ms
loss 20: 50 packets transmitted, 34 received, 32% packet loss, time 50155ms
reorder 25: 50 packets transmitted, 50 received, 0% packet loss, time 49134ms
reorder 50: 50 packets transmitted, 50 received, 0% packet loss, time 49254ms
corrupt 5: 50 packets transmitted, 46 received, 8% packet loss, time 50165ms
corrupt 10: 50 packets transmitted, 41 received, 18% packet loss, time 50154ms
combined: 50 packets transmitted, 39 received, 22% packet loss, time 49220ms
*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done

```

Рис. 3.33: Запуск обновленного скрипта

Созданы файлы для всех воспроизводимых экспериментов в каталоге (рис. 3.34):


```

mininet@mininet-wm:~/work/lab_netem$ ./simple-drop.py 10
lab_netem_ii.py ping_combined.dat ping_corrupt_5.dat ping_loss_20.dat ping_reorder_25.dat stats_combined.log stats_corrupt_5.log stats_loss_20.log stats_reorder_25.log
labfile ping_corrupt_10.dat ping_loss_10.dat ping_loss_5.dat ping_reorder_50.dat stats_corrupt_10.log stats_loss_10.log stats_loss_5.log stats_reorder_50.log
mininet@mininet-wm:~/work/lab_netem$ ./simple-drop.py cat ping_reorder_25.dat
101 1
101 2
103 3
104 4
104 5
105 6
102 7
104 8
1072 9

```

Рис. 3.34: Пример одного из файлов воспроизводимого эксперимента

4 Выводы

В результате выполнения данной лабораторной работы я получила навыки работы проведения интерактивных экспериментов в среде Mininet по исследованию параметров сети, связанных с потерей, дублированием, изменением порядка и повреждением пакетов при передаче данных.